

Version Control for Code You Did Not Write

Recording history at the level of what was asked for

Anonymous Author(s)

WHAT THE DEVELOPER SAID

14:02 to 15:20, one sitting

14:02
"rate limit the public API, 60 requests a minute per key"

14:31
"the price lookup is slow, cache it"

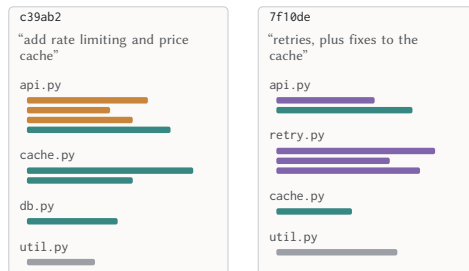
15:01
"retry the upstream call when it returns a 5xx"

nobody asked for this
the agent renamed fetch to fetch_row and fixed its four callers

The price cache turns out to be wrong. Take it back out.

WHAT GIT STORED

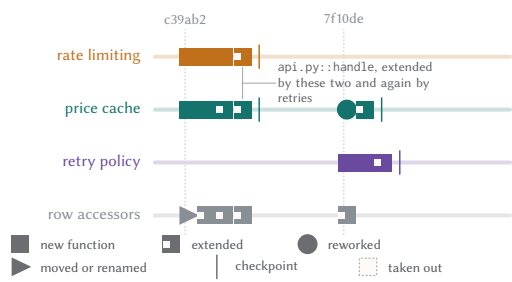
2 commits, 5 files, 15 hunks



Find the 6 cache hunks among the 15, in 3 files, in 2 commits. Split both commits, and check that the fetch_row rename did not ride along with them.

WHAT sgt STORED

18 edits over 14 functions in 5 files



\$ sgt revert "price cache"
removes 6 edits across 4 symbols, 3 files

Figure 1: One hour of work with a coding agent, and the two records it leaves behind. The developer asked for three things and got a fourth (left). Git kept two commits whose fifteen hunks interleave all four requests (middle). The colours in that panel are ours, and git holds no record of which request produced which hunk, so a developer taking one request back out has to decide for each hunk whether it belongs to that request. sgt recorded one edit per function and filed each under the request it came from (right), so the caching work is a set of six edits with a name, and the one function both the rate limiter and the cache touched is visible as such.

Abstract

A developer asks a coding agent for three things in one hour, and fourteen functions in five files come back changed. The developer stated each request in one sentence of English, and the two commits that arrive hold all three requests interleaved with a rename the agent made on its own initiative. Git addresses a history by lines and by commits, so a developer who wants one of those three requests out of it has to name the lines that request produced, and after an agent wrote the code the developer cannot name them. The unit of record and the unit of thought have come apart.

We present sgt, a version control system that records a change as one edit to one function, files each edit under the request it was made for, and keeps the whole record inside an ordinary git repository. A developer names a request to remove it, put it back, compare two attempts at it, or carry it to another branch, and the code that other requests contributed to the same files stays where it is. Every byte of code in the working files came from the developer or their agent, and sgt reads that code rather than writing any of it.

The design costs the developer three things and we state them at the front. Two edits to the same function from the same starting point are recorded as competing versions that a person decides between, including the pair at opposite ends of a fifty line function that git would have merged without comment. The grouping of edits into named features is inferred from which functions change together, so a person corrects it. Code in a language sgt does not parse is versioned whole, at the grain git already uses.

Keywords

version control, coding agents, program comprehension, developer tools, change history

1 Introduction

A developer working with a coding agent describes the change they want in a sentence or two of English, and the agent writes the code. When a developer on one of our projects asked for a price lookup to be cached because it had become slow, the agent added a cache module, gave the database query a parameter for how long a cached price stays good, changed the request handler to consult the cache before the database, and chose the names of the three new functions. Over the following hour that developer made two more requests, and while carrying out the third the agent revised two of the functions it had written for the second. Barke et al. describe programmers in this mode reading what comes back for the shape they expected rather than line by line [3], and among 410 developers surveyed by Liang et al., understanding and trusting generated code was one of the most frequently reported difficulties [36]. The three sentences the developer typed stayed in the chat window, and what entered the repository was eighteen changes to fourteen functions in five files.

A week later the developer wants the caching gone, and to ask for that they have to state it in the terms their version control system understands, all of which are lines and commits. Git records a version as a snapshot of the whole working tree [53], Darc and Pijul record it as a patch that adds and removes lines in a stated context [38, 46], and under either design the smallest thing a history

can name is a line in a file. `git revert` takes back one commit, `git cherry-pick` moves one commit, and a branch points at one commit. The caching the developer asked for is six changes to four functions in three files, spread across the two commits that hour produced, and neither commit holds only those six. Herzig and Zeller found changes serving several unrelated purposes throughout the histories of five open source Java projects [24, 27], and a decade of work on separating such changes after the fact follows from that observation [4, 12, 44, 47, 49].

Separating them by hand means deciding, for each of the fifteen hunks in those two commits, whether it belongs to the caching, to the rate limiting the developer asked for half an hour before the caching, to the retry policy they asked for half an hour after it, or to a rename the agent performed without being asked. Six of the fifteen turn out to be the caching, and the developer then splits both commits so those six stand alone, reverses them, and checks that no function they kept calls a function they just deleted. Biggerstaff et al. called the task of recovering which functions implement a named concern the concept assignment problem [5], and the feature location literature exists because reading code to answer it is expensive [13]. Xia et al. instrumented professional developers and found that roughly 58% of their working time went to understanding code [61].

For as long as developers wrote the lines themselves this work was cheap, because the person doing it remembered writing the code, so the history only had to remind them of a decomposition they already held in their head [34, 48]. An agent leaves the developer nothing to remember. The developer holds the sentence they typed, which stated the boundary of the change exactly, in their own words, before any of the code existed, and which no version control system records. The unit a developer decides in and the unit their repository records no longer match.

The answer a careful maintainer will give to all of this is discipline at commit time. Commit small, commit often, stage hunk by hunk with `git add -p`, and each request ends up with a commit of its own. We have asked contributors on our own projects for exactly this and have held ourselves to it, and it works. It asks the developer to know, at the moment of committing, which lines serve which purpose, and an agent session is exactly the case where they do not. When fourteen changed functions come back and the developer's own contribution was three sentences of English, staging hunk by hunk means reading the whole diff to reconstruct a boundary the agent held and never wrote down. Thimbleby argued that a system should let a person defer a decision until they hold the information the decision needs [52]. Commit hygiene asks for the decision first, and that was affordable for twenty years because the person deciding had just typed the lines.

We present `sgt`, a version control system that records a change as one edit to one function and files each edit under the request it was made for. The developer keeps the editor and the agent they already use, and `sgt` reads what those two leave in the files, so nothing about how the developer works has to change. A save turns the pending changes into one record per changed function, and each record names the function it changed, the version it started from, the version it produced, and the other functions it was built on. `sgt` writes those records into the repository and commits them with the code, so a colleague who has not installed `sgt` pulls an

ordinary git history. Figure 1 shows the hour described above under both records.

Recording at that grain turns the operations the developer already wanted into commands they can type. `sgt revert "price cache"` removes the six edits filed under the price cache and names every function where later work had built on them, and the rate limiting in the same file is byte for byte what it was. `sgt show cache.py: :get_cached` reports which request that function came from, which save reworked it, and how many edits would come out along with it. When the developer decides a week later that the cache was needed after all, `sgt restore` puts the same six edits back, and a database index a colleague added in the meantime is still there, because it was never in the set that came out. None of these commands asks the developer to look at a hunk.

This design costs the developer three things, and we would state them here rather than in a limitations section at the end. Two edits to the same function made from the same starting point are recorded as two competing versions and a person has to decide which one survives, including the case of two edits at opposite ends of a fifty line function that touch no common line and that git would merge without comment. The grouping of edits into named requests is inferred from which functions change together and which call each other, so it is a guess and it will sometimes be wrong, and correcting it costs the developer a rename, a merge, or a split, all three of which move labels and leave every recorded edit alone. `sgt` reads Python, TypeScript, and TSX with tree-sitter [8] and versions every other file whole, at the grain git already uses, so a repository whose interesting logic is written in Go gets nothing from `sgt` today.

This paper makes four contributions. We characterise the mismatch between the unit a developer using an agent decides in and the unit a version control system records, one attempted operation at a time (Section 3), following the style of conceptual analysis Pérez De Rosso and Jackson applied to git itself [41, 42]. We describe `sgt`'s design as four commitments and say what each one buys and what it costs (Section 4). We run the whole set of operations on one repository and give the reasoning behind each command (Section 5). We report what we have learned from `sgt` tracking its own development, and where the design breaks down (Sections 6 and 7).

2 Related Work

2.1 Programming by request

Studies of programming with a language model are four years old, and the unit of work they measure changed inside that span. Barke et al. watched 20 programmers and found them using the model in two ways, finishing a line they had already planned, or asking for a starting point in code they did not yet know how to write [3]. Vaithilingam et al. found that 24 participants using Copilot did not complete tasks faster and still preferred to keep it on, because it gave them a draft to work from [57]. In both studies the developer accepted a line or a block of code. Four years later the benchmark that organises the field gives a model a natural language issue report from a real repository and scores it on whether the repository's own tests pass afterwards [29], and the systems built to score well on it edit several files in a loop [62]. The unit a developer hands

over has gone from a line to a paragraph of English, and the code that comes back is correspondingly larger.

All of this work studies the minutes around acceptance, asking whether the code is correct [43], whether the developer can tell that it is correct [58], and whether they trust it [9, 36], and it stops once the developer has said yes. A week later that same developer wants to remove the caching they asked for and keep the rate limiting they asked for in the same hour, and removing one and keeping the other is a version control operation. We know of no study of what happens to agent-written code after it is committed, and the tools it is committed with were designed for a workflow in which the person committing wrote the lines.

2.2 What a version control system agrees to record

Git records a version as a snapshot of the whole working tree, and each commit names its parents plus a tree of file contents addressed by hash [53]. Verifying that a history has not been altered is then a matter of rehashing it, and a merge is computed on demand from a common ancestor, so no operation in git has to reason about how two edits interact. We would insist on both of those properties in anything we built. Pérez De Rosso and Jackson analysed git's concepts against the purposes users bring to them and found several places where a concept serves a purpose users do not have, the staging area and detached HEAD among them. Their redesign, Gitless, removes both [41, 42]. The commit survives their redesign untouched, and it should, because it serves a purpose users do have. The commit is the smallest thing git can address, so any operation over less than a commit works by having the developer point at lines, which is what `git add -p` and `git rebase -i` are for. Under git's original constraints this is the cheapest correct design available. The person running `git add -p` in 2005 had typed the lines minutes earlier, so the decomposition was already in their head, and recording it in the repository would have meant maintaining a second structure that could disagree with the first. Asking the developer at commit time is free when the developer knows.

Darcs takes the other option and records a version as a patch that adds and removes lines in a stated context [46]. Patches that do not depend on each other commute, so a user can pull the third patch in a series without the first two, and the theory behind this has been formalised more than once [28, 40]. Pijul reimplements it on a data structure designed to make the operations fast [38]. A Darcs user who pulls one patch onto another branch moves the patch that was recorded, along with the dependencies it names. Git's cherry-pick reapplies a diff in a context it was not written against and produces a second commit with a second hash, so the same work has two identities and a later rebase has to be told which of them the developer meant. Darcs works out what a patch depends on from the lines the patch touches, so a dependency that leaves no trace in the text goes unrecorded. When an agent adds a `Retry` class in a new file and edits `api.py:handle` to call it, the two patches share no line, so nothing in the record says that pulling the second without the first yields a file referring to a name that does not exist. Darcs is explicit about this and offers a way to declare a dependency by hand. We would have made the same call in 2003. Computing a dependency between two functions requires parsing the language,

and a version control system that parses is a version control system that works for some languages and not others. That is a price Darcs declined to pay and one we do pay, in the form of a system that reads Python, TypeScript and TSX and treats a Go file as an opaque blob.

Jujutsu and Mercurial both add a record of history's own history. Jujutsu gives each change an identifier separate from its commit hash, so rewriting a commit produces a new commit id while "the change ID generally remains the same" [60], and a change with more than one visible commit is called divergent. Conflicts are recorded in commits so that no command fails because of them, and rewriting a commit automatically rebases its descendants [59]. Mercurial's changeset evolution records obsolescence markers, stored apart from changeset data, that say which changeset superseded which, and the wiki describes the result as "a history of changeset modifications that is orthogonal to the repository history of file modifications" [51]. Both designs let a developer say "the change I was working on yesterday" after five rebases and be understood, and both let two people rewrite the same region of history and have the disagreement recorded, so the resolution is a decision somebody makes on purpose. sgt's treatment of two competing versions of one function is the same construction, and we take Jujutsu's divergent change and Mercurial's content-divergent changeset as evidence that recording the disagreement is the right response to it. We part company on what receives the durable identity. In both systems it is a commit, so the identity is exactly as coarse as the commit, and after a developer squashes a two-hour agent session into one commit, one change id covers all three requests and nothing smaller has a name. Peritext makes the same choice outside version control, giving formatting marks in collaborative rich text their own representation, because a record whose unit is the character gets the boundary of a bold span wrong when two people edit it at once [33, 37]. Choosing a unit of record is choosing which questions the record can answer.

2.3 Recovering a decomposition after the fact

A commit that serves several purposes at once is a known problem, and its consequences have been measured. Herzig and Zeller examined the histories of five open source Java projects, found changes addressing more than one concern throughout them, and showed that separating those changes alters which files a defect prediction model identifies as the most likely to contain bugs [26, 27]. The untangling tools built after that finding partition a diff using definition and use relationships between the changed statements [4], use the fine-grained edit events an IDE can record [12], build a dependency graph across the versions a commit spans [44], or present a partition to the developer for correction [47, 49], and they are evaluated on whether they reproduce a grouping that a human labelled. Every one of them is solving a harder problem than sgt solves, because they run after the information is gone. The developer who typed three sentences into an agent stated the boundary out loud before any of the code existed, and the only reason it is absent from the repository is that nothing wrote it down. We have not improved on their inference, and we do not have to, because the constraint this whole line of work was designed around has loosened.

sgt does depend on two results from this literature. Keeping a function's identity when it is renamed or moved is the problem that refactoring detection and tree differencing address [14, 15, 56], and it is the weakest link in our implementation, since a rename we fail to detect appears as one function removed and another added. Grouping edits into features is a clustering over which functions change together and which call each other, which is the signal the co-change mining literature established [18, 32, 63, 67], resolved with a community detection algorithm [54, 55]. Both are inferences, and Section 4 states what a developer does when they are wrong.

2.4 Histories the editor keeps

A second tradition puts the record in the editor and asks nothing of the developer. Azurite stores a programmer's editing operations at a finer grain than any commit and lets them undo one past edit while keeping the edits made after it [65, 66], which is worth building because backtracking is frequent enough to be part of ordinary programming [64]. Variolite lets a data scientist wrap a region of code as a variant and switch between variants in place, and Kery et al. built it after finding that people doing exploratory work avoid version control for exactly that work [30]. Related systems keep alternative program versions side by side [23, 50], replay a document's editing history [21], version at the grain of a single experiment [39], and help a person clean up a notebook after the fact [25, 31]. In all of these systems the record is a byproduct of working and the developer is never asked to prepare it, and sgt keeps that commitment. All of these systems stop at the boundary of one machine. The history lives in the editor's own store, so a colleague, a build, and a code review never see it, and the unit is a single edit event, so a set of related edits still has no name a person can use. sgt pays for crossing that boundary by writing its record into the git repository as files committed with the code, and Section 4 describes what that costs when a colleague who has not installed sgt commits to the same branch.

2.5 Layers over git built for the way people work now

The tools developers reach for today accept git's storage and add a layer above it. GitButler keeps several branches applied to one working directory at once, draws each as a vertical lane with its own staging area, and lets the developer drag a change into the lane it belongs to [16]. A developer should not have to decide which branch a piece of work belongs to before starting it, because that decision is easiest once the work exists, and we agree with GitButler completely on this. GitButler assigns an individual file change to a lane, so it cannot express the case at the centre of Figure 1, where three requests all edited `api.py::handle` and no assignment of `api.py` to one lane is correct. Graphite and GitHub's stacked pull requests take the ordered series as their unit. A stack is a sequence of changes in which each one targets the layer below it, and the tooling keeps the chain consistent, rebasing and retargeting the higher layers when a lower one merges and landing every unmerged layer beneath the topmost ready one in a single operation [17, 19]. A reviewer cannot read one large change with the care they can give to each of five small ones, and stacking is the right answer to that. Graphite's guide also states the precondition plainly, saying that

developers must understand how to break down their work into small, incremental changes, and every command the tool offers operates on a decomposition the developer has already performed. The agent session is the case where that decomposition exists in the developer's request history and nowhere in the code.

The closest system to ours is sem, which parses about 32 languages with tree-sitter and reports which functions, methods and classes a commit changed, resolving an entity's identity across versions in three passes, by exact identifier, by a structural hash that ignores whitespace and comments, and by token overlap above 80% [1]. Its commands answer questions about entities. `sem impact` lists an entity's dependencies, dependents and tests, `sem log` surfaces which entities change most often and which change together, and `sem context` assembles a bundle about an entity for a model within a token budget. `sem` keeps all of this in a derived SQLite cache outside the repository, under the operating system's cache directory, and that placement is the design decision we want to draw attention to. It makes `sem` safe to adopt and safe to abandon, gives entity-level answers about a history recorded years before `sem` existed, and requires agreement from nobody else on the team. It also bounds what `sem` can offer, because a record any teammate's ordinary commit can silently invalidate cannot be the record an operation acts on, and there is nowhere to write the outcome down. Every `sem` command therefore reports, and none of them changes state. A developer learns from `sem diff` that `get_cached` changed and then removes it with a line-level tool. `sgt` commits its record to the repository in order to act on it, and we pay for that in two places. A colleague without `sgt` installed can merge code and leave the record describing a state that no longer exists, and a record that operations depend on has to be repaired when it is wrong. Sections 4 and 5 are largely about how a developer notices that gap and closes it.

3 Five Things a Developer Cannot Ask For

The five scenarios below come from our own use of `sgt` while building it and from watching colleagues work with coding agents on repositories we maintain. Each one is an operation a developer tried to perform, stated in the words they used, and none of the five is available in git for the same underlying reason. In every case the developer named a set of edits and identified that set by the purpose the edits served, and the smallest thing git's record can name is a line inside a commit (Figure 2). Every scenario has a workaround and all the workarounds work, so our argument is about what they cost.

3.1 "Commit this when it is done, and I do not know yet when that is"

A developer who works through an agent still has to choose when to commit, and in this hour every moment available to them is wrong in a different way. Committing after each request writes a version of `api.py::handle` into the permanent history that existed for twenty-nine minutes and that nobody ever ran the test suite against, and it files the agent's rework of the cache under the `retry` commit, because the agent did that rework while adding the `retry`. Committing once at the end produces a single commit holding all three requests and the rename, which is the state Figure 1

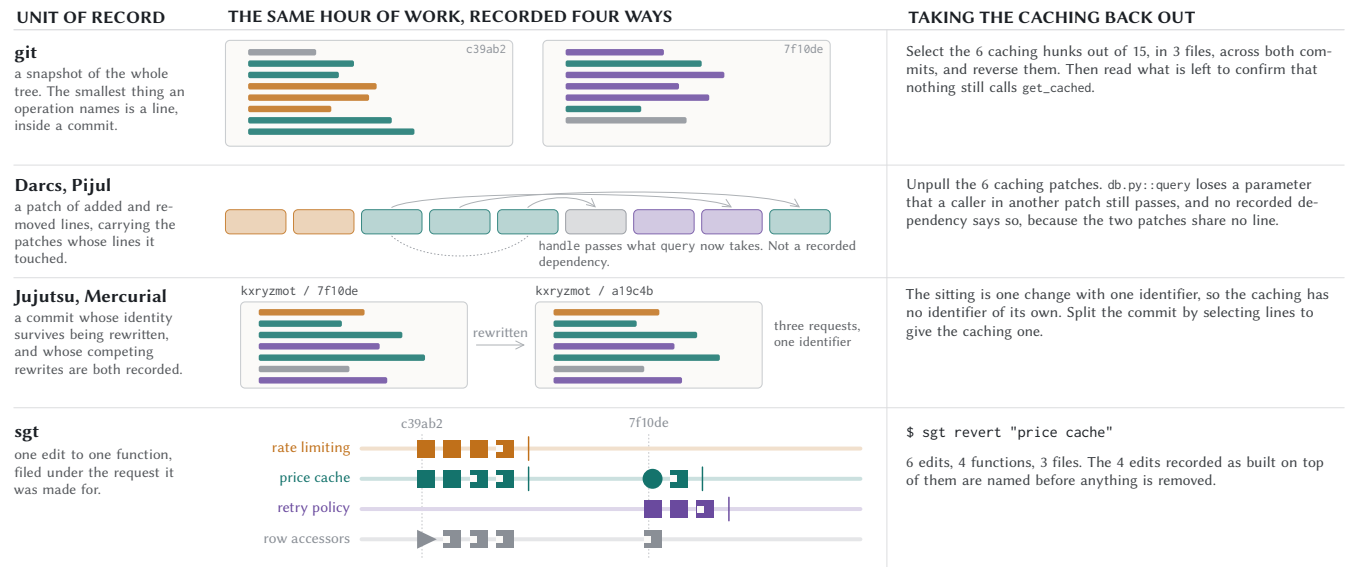


Figure 2: One hour of work with a coding agent, recorded four ways, with the same request removed from each. The unit a system records decides which sets a developer can name, and every row here is a design we think is correct for the constraints it was built under. Git and Darcs both bottom out at a line, so removing the caching means selecting lines. Jujutsu and Mercurial add a durable identity above the line, and it is the identity of a commit, so a squashed session carries one identity for all three requests. sgt records one edit per function together with the request it was made for, which is the same information the developer supplied and never wrote down.

shows. Staging hunk by hunk produces four clean commits and asks the developer to decide, for each of fifteen hunks, which of those four purposes it serves. Green and Petre named these two costs premature commitment, where a notation forces a decision before the information the decision needs is available, and viscosity, the resistance a notation puts up once the decision turns out to be wrong [6, 20]. Git asks for the decomposition at the moment of writing, which in a session driven by requests is a moment when the boundary does exist and the agent is the only party holding it, and the developer's cost of recovering it grows with every later request that touches the same functions.

The operation. Record the boundary the developer already stated, without asking them to state it a second time in a different notation.

3.2 “Change the rate limiter I built yesterday”

The rate limiting from 14:02 used a fixed one-minute window, and three days later it needs a sliding one. The work lands in two of the four functions the original request produced, and the developer's first question is what the rate limiting looked like before they touched it. Git can follow one function at a time. `git log -L :handle:api.py` follows a function through the history by matching a name pattern, and `git log -S` finds the commits where a given string appeared or disappeared. Both take one function or one string at a time, so the developer runs the first command four times, once per function they believe was involved, and assembles the four answers themselves. The set has no name, so nothing checks whether four was the right number. Codoban et al. asked developers why they examine history and found the most common reason to

be understanding the rationale behind a piece of code [11], and LaToza and Myers found questions about why code is the way it is to be among those developers report as hardest to answer [35]. The rationale in this case was three sentences of English, typed three days ago into a window that is now closed.

The operation. Name the work one request produced, see its current state and its previous state, and change it as a unit.

3.3 “Take the caching back out and leave the rest alone”

The caching turned out to serve a stale price for up to sixty seconds, and it has to go. Its six edits are spread over three files and two commits, mixed with nine hunks belonging to three other purposes. One of the six adds a parameter to `db.py::query` and another passes it from `api.py::handle`, and three functions the rename rewrote afterwards call a helper the caching introduced. `git revert` takes back a whole commit, so reverting either of these two removes work the developer is keeping. The path that works is to check out both commits, select the six hunks by hand, reverse them, and then confirm that removing the parameter did not break `handle` and that the three functions the rename rewrote no longer call `get_cached`. That confirmation is where the time goes, because the record contains no statement that the `db.py::query` signature and the `handle` call site were changed for the same reason. Brindescu et al. found that the difficulty of resolving a conflict is predictable from properties of the change, and that how far the change reaches is one of the properties that predicts trouble [7]. A hand-assembled

reverse patch across two commits has exactly the properties their model warns about.

The operation. Remove a named set of edits together with everything recorded as depending on it, and leave every other edit to the same files untouched.

3.4 “Send just the export work to the release branch”

An export bug fix has to ship today, and it was written on a branch that also carries two unfinished features. `git cherry-pick` moves a commit, and the export work is two functions in a commit that also carries part of the unfinished retry policy. The usual answer is a temporary branch and an interactive rebase that splits that commit, and the split has to be done again if more export work arrives before the release. Darcs solves the part of this problem that is about lines, because a Darcs patch carries the dependencies it was recorded with and moving it involves no reapplication (Section 2). The export code calls a helper introduced by one of the unfinished features, and that call is the dependency that decides this case. No line-level record states it, so the developer finds out when the release branch fails to import.

The operation. Carry a named set of edits and its dependencies to another branch, and be told before the operation which dependencies would come along.

3.5 “Let someone review the rate limiter while I keep building on it”

Code review research has converged on small, focused changes, and that finding is why stacked pull requests exist [2, 45]. The rate limiter was finished at 14:30 and the caching that followed calls `api.py::_bucket`, so the two form a stack of two layers with the rate limiter underneath. Submitting them that way required the rate limiter to be on its own branch at 14:30, which required knowing at 14:30 that the caching would build on it. Graphite and GitHub’s stacked pull requests automate every step after that decision, including rebasing the upper layer when a reviewer asks for a change to the lower one (Section 2). The decision itself belongs to the developer, it is made before the work exists, and an agent session is the case where the layering becomes visible only once both layers are written. Church et al. call this a hidden dependency, a relationship the system relies on that the notation does not show, and they make the point about version control specifically [10].

The operation. Discover the layering after the work exists, and submit one layer while continuing to work on the layer above it.

3.6 What the five have in common

Every one of these five operations takes a set of edits as its argument, and the purpose those edits served is what identifies the set. Git expresses a set of lines, a set of files, and a set of commits, and a developer can build all five operations out of those three. The price of building them is recovering a decomposition that nobody recorded, which the untangling literature has been measuring for a decade, and which stayed affordable for as long as the person

doing the recovery was the person who had written the code. Section 4 describes what we changed so that a developer can type the argument these five operations want.

4 What sgt Commits To

sgt records one edit per changed function and files it under the request the edit was made for, and the other three commitments in this section follow from that one. We state each of the four as the developer meets it, with its price attached, because all four prices arrive in the first hour of use. Figure 3 shows the first two at work on the sitting from Figure 1.

4.1 One edit to one function, filed under the request

A save reads the working files, compares them against the last recorded state, and writes one record per changed function. Each record names the function, the version of it the edit started from, the version it produced, and the other functions the new code refers to. The developer supplies one sentence, `sgt save -m "retry the upstream call when it returns a 5xx"`, and sgt reads everything else out of the files, so a record can be no more wrong than the code it was read from.

We chose the function as the unit because the line and the commit both fail on the same ordinary case. The price cache in Figure 3 added a parameter to `db.py::query` and passed a matching argument from `api.py::handle`. Those two changes share no line and no file. A record that bottoms out at a line has nowhere to write down that the two were made for one reason, so the developer who removes the cache a week later learns about the second one when the import fails. A record whose unit is the commit holds whatever was pending when the developer typed the command, which in Figure 1 is three requests and a rename the agent did on its own initiative, divided across two commits by the clock. A function is the smallest thing that both the developer and the agent name out loud while the work is happening, and it is the unit the feature location literature settled on for a related reason, that a concern is a set of functions and almost never a set of lines [5, 13].

sgt finds functions in Python, TypeScript, and TSX with tree-sitter [8] and records every other file as its whole content, at the grain git already offers, so a repository whose interesting logic is in Go gets one record per file and none of the operations below become any sharper than the ones git offers today. Within the three parsed languages, a function can be smaller than a purpose. The rename in Figure 1 moved `db.py::fetch` to `fetch_row` and updated four call sites, and sgt writes five records for what the developer thinks of as one action. Those five records are one named group, so the mismatch shows up only when a developer reads the edit list underneath the name. A function can also be larger than a purpose, which happens when two requests change opposite ends of one fifty line function, and Section 4.3 describes what sgt does then.

4.2 A version is a set of edits, and the files are rebuilt from it

Two rules decide which sets are versions. No edit is included without the edits it was built on, and each function has one included version. sgt rebuilds each file by writing out the included version of



Figure 3: What one save writes down, what a version is, and where the bytes in a file come from. A save writes one record per changed function and files the records under a guess at the request they served, which the developer corrects with one command (left). A version is the set of those records, so taking out the price cache is subtracting six records from eighteen, and two rules decide which sets are allowed (middle). The files are rebuilt from the set with the text between functions kept exactly, so the same set always produces the same bytes (right). Two kinds of mark in the right-hand column are the two consequences sgt reports about this revert and does not act on. The hollow triangle on `api.py::handle` means the retry policy edited that function after the cache did, so sgt took the cache's lines back out of the current text and the developer's test suite decides whether the result stands. The open circles are three functions that still call `cache.py::get_cached`, which sgt names and leaves alone. `db.py::query` is the one function that goes back to text it had earlier in the day.

every function it contains and keeping the text between functions exactly as it was, which produces the bytes that are checked out. The middle panel of Figure 3 is one such set before and after a revert, and the right panel is the file each set produces.

Because a version is a set, revert, restore, diff, and switch all take a set and return a set, and two properties follow that a developer can check on their own repository. Reverting the price cache and then restoring it produces a file byte-identical to the one that was there before either command ran, because both operations are set arithmetic over the same records and neither reapplies a patch. Each of the four can also be shown in full before it happens, because working out the resulting files takes no more than rebuilding them from the proposed set. `sgt revert "price cache"` draws the twelve edits that would remain, names the functions whose text would change, and then asks `apply this revert? [y/N]`. Run with no terminal attached it prints the same preview and declines, so a revert nobody watched does not happen.

Three requests wrote `api.py::handle`, and in Figure 3 it carries an edit from the rate limiter, one from the cache, and one from the retry policy, in that order. Removing the cache's edit under the first rule alone means removing everything recorded above it in that function, and the retry policy is above it. We shipped that behaviour first. On a history where features interleave inside a shared function it removed a later feature in order to remove an earlier one, with a preview that counted edits and therefore looked reasonable, and we

changed the default in August 2026 after it emptied a test repository we were using for exactly this measurement. The default now keeps the later edits and subtracts the reverted feature's lines from the current text of the shared function with a three-way merge, the same operation git performs when it merges a text file. The version of `api.py::handle` that Figure 3 leaves in the file is produced that way, and it is marked. The old behaviour is still available as `revert --take-dependents`, and its help text says what it removes.

That splice is the one place sgt changes text that no person wrote, so it is also where we are most careful about what it declines to do. When the subtraction overlaps a line a later edit changed, sgt leaves the function exactly as it is and prints `kept unchanged` (the removal overlaps later edits—needs your edit) with the function named. When a function sgt is keeping still calls something the revert removed, it prints `still references removed code (fix or revert separately)`. Both of these are reports and neither stops the operation, which is the stance sgt takes throughout. Running `sgt log --summary` in our own repository today prints 279 file(s) on disk differ from the recorded state—'sgt save' absorbs them, and every command still works while that is true. The honest cost of the default is that after a splice the shared function is text the developer has not read, and their test suite is what decides whether the revert was correct. We think that is the right place for the decision, and naming the function before the revert runs is as far as we are willing to go on the developer's behalf.

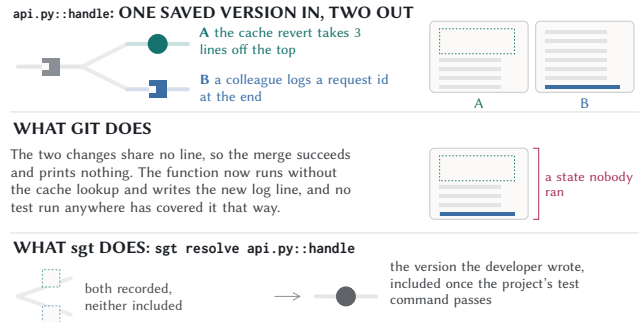


Figure 4: Two people change `api.py::handle` from the version both of them last saved, one taking three lines off the top of it and one adding a line to the logging block at its end. Git's merge of the two succeeds and prints nothing, and nobody has run the function it produces in that state. sgt records both versions, includes neither, and names the function, which is the only refusal in sgt that can stop work that would have been correct.

4.3 Two versions of one function, and a person decides

When two edits change the same function from the same starting version, sgt records both, includes neither, and asks the developer. Of the four commitments this one costs the most goodwill in the first week, so we want to be precise about whose behaviour we are departing from and why theirs is right.

Git merges text and knows nothing about what a function is, which is the correct choice for a system that has to work on every text file in every language anyone will ever write, and it is why git merge is useful on a Makefile. In the case Figure 4 draws, two people had last saved the same version of `api.py::handle`. One of them reverted a caching feature, which took the cache lookup out of the first three lines of that function, and the other added a request identifier to the logging block at the end of it, forty lines away, with no line in common. Git merges the two without comment, and the function that comes out of the merge has never been run in that state by anybody. sgt sees one function with two versions from one parent, names it, and stops. `sgt resolve api.py::handle` prints the two versions beside each other and asks `pick [l]eft / [r]ight / [e]dit / [q]uit`, where the third choice opens the file in the developer's editor and the fourth leaves the fork open. Whatever they write has to pass the project's test command before sgt will include it. Where no test command is configured the developer supplies the verdict with `--override pass` and a `--reason`, and the reason and their name are recorded beside the resolution, so the next person to read that function finds out why it looks the way it does.

A developer pays for this by being stopped in cases where the merge would have been fine, and in one of those cases sgt stops for nothing at all. Deleting a function and adding it back with byte-identical content in two places produces two versions from one parent, and sgt treats content that agrees exactly as a disagreement. Our position is that the cost of being asked is bounded and visible

at the moment it is paid, and the cost of a silent textual merge inside one function is a failure that surfaces later somewhere else. We hold that loosely. A developer who merges thirty branches a day would reasonably weigh the two the other way, and for them the current design is worse.

4.4 The names are inferred, and correcting one moves labels only

Every figure in this paper labels a group of edits with a name like *price cache*, and those names are the part of sgt that guesses. Functions that changed in the same saves and that refer to one another are grouped together, using the change coupling signal established by Zimmermann et al. and Ying et al. [63, 67] and a community detection method over the resulting graph [55]. A language model then reads the saves in a group and writes the label. Membership is computed from the code and the label is written from the words, and those two steps fail in different ways, so we treat them separately.

We considered asking the developer to confirm the grouping at save time and decided against it, because answering that question takes the same knowledge commit hygiene takes. A developer who could reliably answer it could also have staged the hunks. The design question we care about is what a wrong guess costs, and correcting a wrong guess moves labels and touches no recorded edit. `sgt feature rename 001f73c2 "price cache"` renames a group whose identifier stays what it was, `sgt feature regroup` moves functions between groups, and `sgt save --as "price cache"` files a new save into a group that already exists. Because identifiers survive all three, a revert written down in a code review comment last month still removes the same edits after somebody renames the group today.

Two failures are worth naming because a reader will hit them. A function that everything calls, an argument parser or a logging helper, changes in nearly every save and pulls otherwise unrelated work into one group, and the fix is a regroup the developer has to notice they need. A save with no message gives the language model less to read, and sgt says so at the time, printing `· no words captured` under the save it just wrote. The label on a group summarises what the developer said about the work, so a developer who says nothing gets a name derived from the code alone.

4.5 What sgt will not do

sgt writes no code of its own. Every byte in the working files came from the developer or their agent, apart from the three-way merge splice in Section 4.2, which reports every case it cannot perform cleanly. We have been asked more than once during this work why sgt does not hand the shared function to a language model and have it redrafted after a revert, and our answer is that a version control system whose output nobody wrote is a version control system nobody can audit.

sgt also does not stand between a developer and git. The records are files under `.sgt/` committed alongside the code, `sgt git <args>` passes through to git untouched, and a colleague who has never installed sgt clones an ordinary repository with an ordinary history, and there is no server and no background process. Two of sgt's commands refuse to run. `switch` stops when there are unsaved edits, because sgt has nowhere to put them, and it names the files and

prints record them with ‘sgt save’, or commit / ‘git restore’ those files, then re-run. restore stops when putting a set back would leave two versions of one function live, and there its message names the edits by identifier. The function whose two versions caused the refusal goes unnamed, which is a rough edge we have not fixed. Everything else sgt finds wrong it reports, and then it proceeds.

4.6 The five operations, and what a developer types

Figure 5 lists every operation sgt offers with its cost beside it. The five operations Section 3 ended on map onto them directly. Recording a boundary without declaring it is `sgt save -m`, run whenever the developer feels like running it, since the grouping is recovered afterwards from the code. Changing the work one request produced is `sgt show "rate limiting"` for every function it covers and `sgt show "rate limiting@1"` for the second stretch of work on it alone, with the edits that stretch made and what removing only that stretch would take with it. Removing a named set is `sgt revert "price cache"`, which prints its consequences first. Carrying a named set to another branch is `sgt propose land --subset "export"`, which declines a subset that omits a group another chosen group depends on and names the group that is missing. Discovering the layering is reading the recorded dependencies, since the cache edits record that they were built on `api.py::_bucket` and therefore on the rate limiter, which is the fact the developer needed at 14:30 and did not have. Section 5 runs these in order on one repository.

5 One Repository, Five Sittings

The repository here is a five thousand line Python service that answers price quotes for a trading dashboard, and the first sitting is the hour Figure 1 draws. We assembled the sequence from sessions we ran while building sgt so that one repository contains the five operations Section 3 asked for, in that order, followed by the one case where sgt stops and asks. Every command below runs and every block of output came out of sgt as it is today. Output lines too wide for this column are wrapped onto a second line and are otherwise unchanged. Read the sequence for which decisions the developer is asked to make, and when in the work they arrive.

5.1 Monday, 14:02 to 15:20: two saves and four names

The developer made three requests in that hour, and the agent renamed a function on its own initiative. At 14:33, with the rate limiting finished and the price cache first drafted, the developer saved.

```
$ sgt save -m "rate limiting and a price cache on quotes"
```

```
✓ save c39ab2f "rate limiting and a price cache on quotes"
├─o new feature (4a1c9e02) — unnamed; name it:
│   sgt feature rename 4a1c9e02 "<label>"
│   api.py::_bucket, api.py::_key, api.py::_window +1 more
├─o new feature (001f73c2) — unnamed; name it: . . .
│   cache.py::get_cached, cache.py::_ttl, db.py::query +1 more
├─o new feature (9d20b18a) — unnamed; name it: . . .
│   db.py::fetch_row, api.py::export_row, util.py::load_row +1 more
└─o one save touched 3 features — deliberate? 'sgt log --map'
    shows them; 'sgt feature regroup move' re-files work
✓ reverse this save: sgt undo
```

The developer typed one sentence and sgt wrote twelve records. The three groups have no names yet, because sgt writes records when the developer saves and works out the names when the developer next reads, and a name costs a language model call that has no business running while somebody is mid-session. The warning is the one line here that resembles what git asks for, and sgt printed it after the save was already written. The developer read it, agreed the sitting had been mixed, and carried on, which is the case Section 3 opened with.

The second save at 15:20 covered the retry policy and a rework of the cache the agent made while adding the retry. By then the first save's groups had names, so its echo listed • Price Cache (001f73c2) for the group that already existed and o new feature (7c02b4d1) for the retry work. `sgt log --tree` afterwards printed the four groups the hour produced.

```
Rate Limiting (4a1c9e02) · 4 symbol(s)
Price Cache (001f73c2) · 4 symbol(s)
Retry Policy (7c02b4d1) · 3 symbol(s)
Row Accessors (9d20b18a) · 5 symbol(s)
```

Three of the four names are what the developer would have written. The fourth is the agent's unrequested rename, named from the code because the developer never said anything about that part of the work, which is the failure Section 4.4 predicts. It is also the only one of the four whose name nobody needs, so nobody corrected it.

5.2 Thursday: changing the rate limiter

The fixed one minute window has to become a sliding one, and the developer starts from what the rate limiting is now.

```
$ sgt show "Rate Limiting"
feature 4a1c9e02 "Rate Limiting"
4 edits · 4 symbols in 1 file · last touched 3d ago
symbols api.py::_bucket, api.py::_key, api.py::_window,
api.py::handle
saves c39ab2f rate limiting and a price cache on quotes
reverting this removes 11 edits — 7 of them work built on top
```

The four functions are the answer to the question that took four runs of `git log -L` in Section 3, and nothing here required the developer to guess that four was the right number of places to look. The developer did not ask for the last line of that output and read it anyway, and it says that seven later edits are recorded as built on this rate limiting, so a sliding window is a change underneath work that already depends on it, and the developer knows that before opening the file.

The agent rewrites `api.py::_window` and `api.py::_bucket` and leaves the other two functions alone, and the developer saves, naming the group this time because they already know which one it is.

command	what the developer gets	what it costs
sgt save -m "..."	One record per changed function, filed under a name for the request.	The grouping is a guess. When it is wrong the developer moves labels, and no recorded edit changes.
sgt show api.py::handle	The request this function came from, the saves that touched it, and how many edits removing it would take along.	The answer is exactly as good as the grouping behind it.
sgt revert "price cache"	The named set subtracted from the version, with every function it changes named before anything is applied.	A function later work also edited has the set's lines subtracted from its current text, and the developer's tests decide whether the result stands.
sgt restore "price cache"	The same set back, byte for byte, with everything added since left alone.	Refused when putting it back would leave two versions of one function live, and the refusal prints edit identifiers with no function name attached.
sgt diff main feat/export	The edits each version has that the other does not, in both directions.	It answers in functions, so seeing which lines moved still means opening the file.
sgt switch feat/export	The working files rebuilt from that branch's set.	Refused on unsaved edits, because sgt has nowhere to put them yet.
sgt intent list	Each feature cut into the stretches of work done on it, each with a name like rate limiting@2 that the other commands accept.	The cuts come from a language model reading the saves, so a stretch can start one save too early.
sgt resolve api.py::handle	The two competing versions of the function side by side, and whichever one the developer picks or writes is included once the project's test command passes.	Two edits at opposite ends of one function are also competing versions, and git would have merged them without asking.
sgt plan intake "..."	A name for work that does not exist yet, which the next save matches itself against.	A plan that predicts whole files matches nothing. It has to name functions.
sgt propose land --subset "export"	One finished feature sent to the shared branch, with the unfinished one left behind.	Two functions that sit next to each other in one file can lose the blank line between them.

Figure 5: The ten operations, each drawn as the set before the command and the set after it. One line is one request and the marks on it are the edits filed under it, as in Figure 1, and a dotted outline is an edit the version no longer includes. No operation takes a hash. Every argument is a sentence the developer wrote, a branch name, or a function name. The right-hand column says what each operation costs, and we put it beside the command because a developer meets these costs on the first day.

```
$ sgt save --as "Rate Limiting" -m "sliding window, 60s of buckets"
✓ save 1f0aee3 "sliding window, 60s of buckets"
└─ Rate Limiting (4a1c9e02) api.py::_bucket, api.py::_window
✓ named "Rate Limiting"
```

--as is the one place a developer states a grouping by hand, and it changed nothing here, because both functions were already in that group and the edits would have been filed there without it. They typed it because they knew the answer and had no reason to let sgt work it out. The rate limiting now has two stretches of work on it, and sgt log --focus 4a1c9e02 names both.

```
• Rate Limiting 4a1c9e02 · 2 checkpoint(s)
[0■■■] 4a1c9e0200 :rate-limiting Rate Limiting
(1■■■) 4a1c9e0201 :sliding-window Sliding Window
operate: sgt revert 4a1c9e02:sliding-window
(one checkpoint, by name) ·
sgt revert 4a1c9e02 (whole feature)
```

The developer never asked for these two stretches. sgt cut the work where their words changed, which in this repository is where one request ended and the next began, and either half is addressable on its own, so sgt revert "Rate Limiting:sliding-window" takes Thursday's work off and leaves Monday's four edits in place. The last two lines exist because we watched people on our own team discover that a stretch of work had a name and then stop, not knowing which command would accept it.

5.3 The following Monday: taking the cache out

The cache serves a stale price for up to sixty seconds and has to go.

```
$ sgt revert "Price Cache"
[the log region redrawn, with the 6 removed edits marked]
subtracted from shared code (later work kept): api.py::handle
removed going forward: cache.py::get_cached, cache.py::_ttl,
db.py::query
△ still references removed code (fix or revert separately):
api.py::export_row, util.py::load_row, util.py::save_row
apply this revert? [y/N]
```

The retry policy edited api.py::handle after the cache had edited it, so that function keeps the retry work and loses the cache's three lines, which is how removing a week old request leaves a change made yesterday in place. Two functions existed only for the cache and go away with it, which empties cache.py and removes the file, and db.py::query loses the parameter the cache added and goes back to the text it had at 14:02. The three functions in the last line are row accessors the agent rewrote during its rename, and it pointed all three at get_cached while it was in there, so their edits belong to a request the developer is keeping and name a function the revert deletes. sgt names them and proceeds, because whether the repair is deleting three call sites or reverting the rename as well is not a question about the recorded edits.

The developer answered y and ran the test suite. It failed in the three places the report named and nowhere else, and the repair was deleting the cache lookup from each of them. A revert can be taken back exactly, and that property is why we build every one of these operations as set arithmetic, so the developer tested it.

```

1161 $ sgt restore "Price Cache"
1162 ✓ restore applied — 0 edit(s) removed, 6 added.
1163 ('sgt undo' reverses this.)

```

Putting the cache back produced the same bytes that were in the files before the revert, and it kept the three call site repairs the developer had made in between, because both directions are set arithmetic over the same records and neither one reapplies a patch. They had wanted the cache gone, so they reverted it a second time, and that run printed the same three consequence lines as the first.

5.4 Wednesday: shipping the export fix alone

A bulk export endpoint has to ship to the release branch today, and the branch it sits on also carries the retry policy, which is half finished. The export code goes into a new `export.py` and applies the rate limit before streaming, so `export.py::write_csv` calls `api.py::_bucket`. Nobody thought about that call as a dependency while writing it.

```

1178 $ sgt propose create --base release
1179 ✓ proposal 3f9a2c81b04d (base release, +17 op(s), 4 feature(s))
1180 $ sgt propose land 3f9a2c81b04d --subset "Quote Export"
1181 × 'Quote Export' requires Rate Limiting -- include it in
1182 --subset too

```

The developer asked to move one group and `sgt` named the group that has to come with it, which is the fifth operation from Section 3. The record of the call to `api.py::_bucket` was written at the moment the export code was saved, and `sgt` reads it now to answer a question nobody was thinking about then. Git would have taken the cherry-pick and the release branch would have failed to import.

```

1189 $ sgt propose land 3f9a2c81b04d --subset "Rate Limiting" "Quote
1190 Export"
1191 ✓ propose land release: 8b21c0d4e7fa (+8 op(s))

```

Two groups went to the release branch in the order the records put them in, and the half finished retry policy and the accessor rename stayed behind. Those two groups are the stack from Section 3, assembled without the branch that had to exist before the work did.

5.5 Thursday: two versions of one function

A colleague on this repository had been editing `api.py::handle` as well, adding a request identifier to every outbound log line. The developer syncs.

```

1202 $ sgt sync
1203 Δ sync origin/main: merged 4f2a1c8b93de
1204 +7 op(s)
1205 Δ 1 OPEN FORK(S) — forked symbol(s) sit at the common
1206 ancestor until resolved:
1207 api.py::handle: sgt merge-op 3c91ab02 8ee14d77

```

That last line names `sgt merge-op`, which takes two edit identifiers and is not the command a person should be reaching for. `sgt log` in the same state prints `api.py::handle → sgt resolve api.py::handle`, which is the command to use. Two commands telling the developer different things to type is an inconsistency we have not cleaned up. The two versions of `api.py::handle` are the one the cache revert produced, three lines shorter at the top where the cache lookup had been, and the colleague's, one line longer inside the logging block at the end. No line is common to the two changes, so git merges them without comment, which is the case Figure 4 draws. `sgt` prints them beside each other and asks.

```

— fork on api.py::handle — (◀ left = tip A ▶ right = tip B)
[the two versions of the function, side by side]
pick [l]eft / [r]ight / [e]dit / [q]uit:

```

The developer picked `e`, merged the two by hand in about thirty seconds, and `sgt` ran the project's test command before including the result. We are not going to claim the textual merge would have been wrong here. It would most likely have been right, and `sgt` charged thirty seconds and one interruption to establish that. The case we designed for is the one where the merge is wrong, where one edit adds an early return and the other edit's new code sits below it, and the test run that would have caught it happens on a branch nobody has pushed yet. A developer who merges often and reads carefully is paying us for a certainty they already had.

5.6 What this cost

Across five sittings and one fork the developer made three decisions `sgt` could not make for them. They decided the export endpoint should ship without the retry policy, they decided how the two versions of `api.py::handle` should read, and they decided that the three functions still calling the cache should lose that call and keep the rest of the agent's rename. All three are decisions about the code, which is where we wanted them.

The developer paid for those three decisions in three places. One group of five edits carries a name derived from the code, because nobody said anything about that part of the work. The revert left `api.py::handle` holding text nobody had read, so their test suite is what decided the revert had worked. The fork stopped them on a merge that was probably fine. The second and third of those are direct consequences of Section 4.2 and Section 4.3, and on a repository this size we would choose both again. On a repository where twenty people edit the same functions each day, the fork rule is the first thing we would expect to have to change.

6 Evaluation

[This section is in preparation and is not part of this submission draft.]

7 Where the Design Breaks

Every number in this section comes from running `sgt` on the repository that contains `sgt`, today, with the command a user would run. We have been the heaviest users of this system for eight months, and the six limits below are what we would want to know before adopting it.

7.1 Most of a repository is not functions

`sgt log --summary` prints 371 file(s), 6781 symbol(s), 154 feature(s), 27% entity coverage. That last figure is the fraction of recorded files with any function-level record at all, so roughly three quarters of this repository is recorded as whole files, at the grain git already offers. The files in that three quarters are Markdown, JSON, HTML, and lock files. Every operation in Section 4 runs on them and none of them gets sharper than `git revert` would be.

Adding languages does not fix this, and we want to be clear about which part of it they do fix. A tree-sitter grammar plus a decision about which node kinds count as a function is a day of work per language, and we would take Go and Rust that way tomorrow. The case that remains after that is a repository whose interesting change

is four lines of a deployment configuration, where the change a developer names has no function to be filed under, and git's grain is the correct grain for it. We say this plainly because calling the design language-independent would be false, and a reader deciding whether to adopt sgt should start by counting how much of their repository is code.

7.2 Where a function loses its history

Section 4.1 claims a record survives a rename, and that claim rests on a matcher with a threshold in it. sgt links a function across a commit first by its name, then by identical bytes, then by identical structure with formatting and comments ignored, and last by token overlap above 0.80, with a guard that rejects any pair whose token counts differ by more than a factor of two. The 0.80 comes from sem, which uses the same figure for the same job [1]. A rename that also rewrites about a fifth of the function falls below it and is recorded as one function removed and a different function added.

The developer then sees a function whose recorded history begins at that commit, with its earlier edits still filed under a group the function is no longer part of. An agent asked to clean up a module produces exactly this, renaming six functions and rewriting their bodies in one sitting. Origin analysis is a well studied problem, and the accurate detectors work from a full syntax tree comparison [14, 18, 32, 56]. sgt uses the token count because it runs on every save and has to finish while the developer is waiting. Lowering the threshold would link functions that merely resemble each other, and a wrong link is the worse failure, because it files one function's edits under another function's name and gives the developer no reason to doubt the answer. sgt identity join states a link by hand, and it requires the developer to notice that a history restarted, and noticing that is what they were using sgt to avoid having to do.

7.3 Fifty-six files we cannot rebuild

The same command reports 56 files whose bytes on disk no valid set of records can produce. sgt keeps every one of them, prints the count, names the first five, and points at sgt advanced fsck --tree. Nothing was deleted and nothing was quietly wrong, because sgt reports what it cannot do instead of refusing to run (Section 4.2). The developer loses the record for those files. A revert of a group that touched one of them leaves the file as it is, and sgt show on a function inside it answers from an incomplete chain.

We have not tracked down what put all 56 in that state. This repository has been mined by five successive versions of our miner, and some of these files are the residue of our own format changes. A developer adopting sgt on a repository with two years of history should expect a number of this kind on the first run, and should read it as work the design has not finished.

7.4 The gate nobody switched on

Section 4.3 says a resolved fork has to pass the project's test command before sgt will include it. The summary above also prints oracle: unconfigured. In the repository we use every day, resolving a fork means the developer picks a version and sgt takes their word for it, because nobody ever pointed sgt at our test runner.

That one line of output is the most uncomfortable in this section, and we have kept it here because the failure it describes generalises past our repository. A safety property a developer has to configure is a safety property most repositories will not have, so the set of repositories where our strongest claim about forks holds is the set where somebody did the setup, and on the evidence of our own it is smaller than we assumed. The fix needs no research, because sgt could find the test command the project already uses and offer it during sgt init, and making the developer ask for it was the wrong default.

7.5 Who should turn the fork rule off

We designed the fork rule for a repository with three contributors, where a merge nobody reviewed turns into a bug three days later and the person who finds it is the person who wrote both sides of it. On a repository where twenty people edit the same functions every day, the same rule produces a queue of decisions nobody has time to work through, and the honest prediction is that they would switch it off and take the textual merge. sgt also stops a developer for nothing at all in one case. Deleting a function and adding it back with byte-identical content on two branches produces two versions from one parent, and sgt reports a disagreement between two pieces of text that agree exactly.

The setting we would build merges when the two edits changed no overlapping lines and asks when they did, which is git's rule applied at the grain of a function. Guimarães and Silva argue for telling a developer about a divergence while both sides are still being written [22], and that setting would let sgt do it without stopping anybody. We have not built it, because we cannot yet tell a reader which of the two should be the default. Our answer comes from repositories with three contributors and a shared understanding of the code, and it is the part of this design that a study of larger teams would most usefully overturn.

7.6 The record the agent already had

The grouping sgt recovers by clustering existed before any of the code did. The agent held it while it worked, and every agent we have used discards it when the session ends. A record written by the agent, one entry per request naming the functions it changed, would make the inference in Section 4.4 unnecessary, and it would fix the case in Section 5 where a rename nobody asked for got a name derived from the code.

We did not build that, because we are unwilling to let sgt depend on any particular agent. Reading the working files works with any agent, any editor, and a developer typing by hand, and it keeps working when a vendor changes an output format. A record the agent writes works while that agent is the one being used, and a developer who switched agents mid-project would find their history split into a part with stated groupings and a part with inferred ones. We would take both sources, with the files as the one sgt trusts and the agent's account as a hint that improves a label. Which of the two should be authoritative is the design question we would put to a reader of this paper first.

References

- [1] Ataraxy Labs. 2026. sem: Semantic version control built on Git. <https://github.com/ataraxy-labs/sem>. Accessed 2026.

- [2] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. (2013), 712–721. doi:10.1109/ICSE.2013.6606617
- [3] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111. doi:10.1145/3586030
- [4] Mike Barnett, Christian Bird, João Brunet, and Shuvendu K. Lahiri. 2015. Helping developers help themselves: Automatic decomposition of code review changesets. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*. 134–144. doi:10.1109/ICSE.2015.35
- [5] Ted J. Biggerstaff, Bharat G. Mitbander, and Dallas Webster. 1993. The concept assignment problem in program understanding. In *Proceedings of the 15th International Conference on Software Engineering (ICSE)*. 482–498. doi:10.1109/ICSE.1993.346017
- [6] Alan F. Blackwell and Thomas R. G. Green. 2003. Notational systems: The Cognitive Dimensions of Notations framework. In *HCI Models, Theories, and Frameworks: Toward a Multidisciplinary Science*, John M. Carroll (Ed.). Morgan Kaufmann, 103–134.
- [7] Caius Brindescu, Iftekhar Ahmed, Rafael Leano, and Anita Sarma. 2020. Planning for untangling: Predicting the difficulty of merge conflicts. In *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 801–811. doi:10.1145/3377811.3380344
- [8] Max Brunsfeld and the tree-sitter contributors. 2026. tree-sitter: An incremental parsing system for programming tools. <https://tree-sitter.github.io/>. Accessed 2026.
- [9] Ruijia Cheng, Ruotong Wang, Thomas Zimmermann, and Denae Ford. 2024. “It would work for me too”: How online communities shape software developers’ trust in AI-powered code generation tools. In *ACM Transactions on Interactive Intelligent Systems*. doi:10.1145/3651990
- [10] Luke Church, Emma Söderberg, and Bhargava Elango. 2014. A case of computational thinking: The subtle effect of hidden dependencies on the user experience of version control. *Proceedings of the 25th Annual Workshop of the Psychology of Programming Interest Group (PPG)* (2014).
- [11] Mihai Codoban, Sruti Srinivasa Ragavan, Danny Dig, and Brian Bailey. 2015. Software history under the lens: A study on why and how developers examine it. In *Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 1–10. doi:10.1109/ICSME.2015.7332446
- [12] Martin Dias, Alberto Bacchelli, Georgios Gousios, Damien Cassou, and Stéphane Ducasse. 2015. Untangling fine-grained code changes. In *Proceedings of the 22nd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. 341–350. doi:10.1109/SANER.2015.7081844
- [13] Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature location in source code: A taxonomy and survey. *Journal of Software: Evolution and Process* 25, 1 (2013), 53–95. doi:10.1002/smr.567
- [14] Jean-Rémy Falleri, Flóreal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*. 313–324. doi:10.1145/2642937.2642982
- [15] Beat Fluri, Michael Würsch, Martin Pinzger, and Harald C. Gall. 2007. Change distilling: Tree differencing for fine-grained source code change extraction. *IEEE Transactions on Software Engineering* 33, 11 (2007), 725–743. doi:10.1109/TSE.2007.70731
- [16] GitButler. 2026. GitButler documentation: Parallel branches. <https://docs.gitbutler.com/features/virtual-branches/virtual-branches>. Accessed 2026.
- [17] GitHub. 2026. Stacked pull requests are now in public preview. <https://github.blog/changelog/2026-07-30-stacked-pull-requests-are-now-in-public-preview/>. Accessed 2026.
- [18] Michael W. Godfrey and Lijie Zou. 2005. Using origin analysis to detect merging and splitting of source code entities. *IEEE Transactions on Software Engineering* 31, 2 (2005), 166–181. doi:10.1109/TSE.2005.28
- [19] Graphite. 2026. Stacked diffs and how to use them. <https://graphite.com/guides/stacked-diffs>. Accessed 2026.
- [20] Thomas R. G. Green and Marian Petre. 1996. Usability analysis of visual programming environments: A ‘cognitive dimensions’ framework. *Journal of Visual Languages and Computing* 7, 2 (1996), 131–174. doi:10.1006/jvlc.1996.0009
- [21] Tovi Grossman, Justin Matejka, and George Fitzmaurice. 2010. Chronicle: Capture, exploration, and playback of document workflow histories. In *Proceedings of the 23rd Annual ACM Symposium on User Interface Software and Technology (UIST)*. 143–152. doi:10.1145/1866029.1866054
- [22] Mário Luís Guimarães and António Rito Silva. 2012. Improving early detection of software merge conflicts. In *Proceedings of the 34th International Conference on Software Engineering (ICSE)*. 342–352. doi:10.1109/ICSE.2012.6227180
- [23] Björn Hartmann, Loren Yu, Abel Allison, Yeonsoo Yang, and Scott R. Klemmer. 2008. Design as exploration: Creating interface alternatives through parallel authoring and runtime tuning. In *Proceedings of the 21st Annual ACM Symposium on User Interface Software and Technology (UIST)*. 91–100. doi:10.1145/1449715.1449732
- [24] Lile P. Hattori and Michele Lanza. 2008. On the nature of commits. In *Proceedings of the 23rd IEEE/ACM International Conference on Automated Software Engineering (ASE) Workshops*. 63–71. doi:10.1109/ASEW.2008.4686322
- [25] Andrew Head, Fred Hohman, Titus Barik, Steven M. Drucker, and Robert DeLine. 2019. Managing messes in computational notebooks. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–12. doi:10.1145/3290605.3300776
- [26] Kim Herzig, Sascha Just, and Andreas Zeller. 2016. The impact of tangled code changes on defect prediction models. *Empirical Software Engineering* 21, 2 (2016), 303–336. doi:10.1007/s10664-015-9376-6
- [27] Kim Herzig and Andreas Zeller. 2013. The impact of tangled code changes. In *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*. 121–130. doi:10.1109/MSR.2013.6624018
- [28] Judah Jacobson. 2009. A formalization of Darcs patch theory using inverse semigroups. *Computational and Applied Mathematics Reports, University of California, Los Angeles* (2009). CAM report 09-83.
- [29] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues?. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- [30] Mary Beth Kery, Amber Horvath, and Brad A. Myers. 2017. Variolite: Supporting exploratory programming by data scientists. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 1265–1276. doi:10.1145/3025453.3025626
- [31] Mary Beth Kery, Marissa Radensky, Mahima Arya, Bonnie E. John, and Brad A. Myers. 2018. The story in the notebook: Exploratory data science using a literate programming tool. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–11. doi:10.1145/3173574.3173748
- [32] Sunghun Kim, Kai Pan, and E. James Whitehead. 2005. When functions change their names: Automatic detection of origin relationships. In *Proceedings of the 12th Working Conference on Reverse Engineering (WCRE)*. 143–152. doi:10.1109/WCRE.2005.33
- [33] Martin Kleppmann, Victor B. F. Gomes, Dominic P. Mulligan, and Alastair R. Beresford. 2019. Interleaving anomalies in collaborative text editors. In *Proceedings of the 6th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC)*. 1–7. doi:10.1145/3301419.3323972
- [34] Andrew J. Ko, Brad A. Myers, Michael J. Coblenz, and Htet Htet Aung. 2006. An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks. *IEEE Transactions on Software Engineering* 32, 12 (2006), 971–987. doi:10.1109/TSE.2006.116
- [35] Thomas D. LaToza and Brad A. Myers. 2010. Hard-to-answer questions about code. In *Evaluation and Usability of Programming Languages and Tools (PLATEAU)*. 1–6. doi:10.1145/1937117.1937125
- [36] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. 2024. A large-scale survey on the usability of AI programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 1–13. doi:10.1145/3597503.3608128
- [37] Geoffrey Litt, Sarah Lim, Martin Kleppmann, and Peter van Hardenberg. 2022. Peritext: A CRDT for collaborative rich text editing. *Proceedings of the ACM on Human-Computer Interaction* 6, CSCW2 (2022), 1–29. doi:10.1145/3555644
- [38] Pierre-Étienne Meunier and Florent Becker. 2024. The Pijul manual. <https://pijul.org/manual/>. Accessed 2026.
- [39] Hiroaki Mikami, Daisuke Sakamoto, and Takeo Igarashi. 2017. Micro-versioning tool to support experimentation in exploratory programming. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 6208–6219. doi:10.1145/3025453.3025597
- [40] Samuel Mimram and Cinzia Di Giusto. 2013. A categorical theory of patches. *Electronic Notes in Theoretical Computer Science* 298 (2013), 283–307. doi:10.1016/j.entcs.2013.09.018
- [41] Santiago Perez De Rosso and Daniel Jackson. 2013. What’s wrong with git? A conceptual design analysis. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward!)*. ACM, 37–52. doi:10.1145/2509578.2509584
- [42] Santiago Perez De Rosso and Daniel Jackson. 2016. Purposes, concepts, misfits, and a redesign of git. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. ACM, 292–310. doi:10.1145/2983990.2984018
- [43] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do users write more insecure code with AI assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2785–2799. doi:10.1145/3576915.3623157
- [44] Profir-Petru Părtachi, Santanu Kumar Dash, Christoph Treude, and Earl T. Barr. 2020. Flexeme: Untangling commits using lexical flows. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 63–74. doi:10.1145/3368089.3409693

- [45] Peter C. Rigby and Christian Bird. 2013. Convergent contemporary software peer review practices. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 202–212. doi:10.1145/2491411.2491444
- [46] David Roundy. 2005. Darcs: Distributed version management in Haskell. *Proceedings of the 2005 ACM SIGPLAN Workshop on Haskell* (2005), 1–4. doi:10.1145/1088348.1088349
- [47] Bo Shen, Wei Zhang, Christian Kästner, Haiyan Zhao, Zhao Wei, Guangtai Liang, and Zhi Jin. 2021. SmartCommit: A graph-based interactive assistant for activity-oriented commits. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 379–390. doi:10.1145/3468264.3468551
- [48] Jonathan Sillito, Gail C. Murphy, and Kris De Volder. 2008. Asking and answering questions during a programming change task. *IEEE Transactions on Software Engineering* 34, 4 (2008), 434–451. doi:10.1109/TSE.2008.26
- [49] Yida Tao and Sunghun Kim. 2015. Partitioning composite code changes to facilitate code review. In *Proceedings of the 12th Working Conference on Mining Software Repositories (MSR)*. 180–190. doi:10.1109/MSR.2015.24
- [50] Michael Terry, Elizabeth D. Mynatt, Kumiyo Nakakoji, and Yasuhiro Yamamoto. 2004. Variation in element and action: Supporting simultaneous development of alternative solutions. In *Proceedings of the 2004 CHI Conference on Human Factors in Computing Systems*. 711–718. doi:10.1145/985692.985782
- [51] The Mercurial contributors. 2026. Mercurial wiki: Changeset evolution. <https://wiki.mercurial-scm.org/ChangesetEvolution>. Accessed August 2026.
- [52] Harold Thimbleby. 1988. Delaying commitment. *IEEE Software* 5, 3 (1988), 78–86. doi:10.1109/52.2027
- [53] Linus Torvalds, Junio C. Hamano, and the Git contributors. 2026. Git: Distributed version control system. <https://git-scm.com/>. Accessed 2026.
- [54] V. A. Traag, P. Van Dooren, and Y. Nesterov. 2011. Narrow scope for resolution-limit-free community detection. *Physical Review E* 84 (2011), 016114. doi:10.1103/PhysRevE.84.016114
- [55] V. A. Traag, L. Waltman, and N. J. van Eck. 2019. From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports* 9 (2019), 5233. doi:10.1038/s41598-019-41695-z
- [56] Nikolaos Tsantalis, Matin Mansouri, Laleh M. Eshkevari, Davood Mazinanian, and Danny Dig. 2018. Accurate and efficient refactoring detection in commit history. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. 483–494. doi:10.1145/3180155.3180206
- [57] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. 1–7. doi:10.1145/3491101.3519665
- [58] Helena Vasconcelos, Gagan Bansal, Adam Fournery, Q. Vera Liao, and Jennifer Wortman Vaughan. 2025. Generation probabilities are not enough: Uncertainty highlighting in AI code completions. In *ACM Transactions on Computer-Human Interaction*. doi:10.1145/3702320
- [59] Martin von Zweigbergk and the Jujutsu contributors. 2026. Jujutsu documentation: Comparison with Git. <http://docs.jj-vcs.dev/latest/git-comparison/>. Accessed August 2026.
- [60] Martin von Zweigbergk and the Jujutsu contributors. 2026. Jujutsu documentation: Glossary. <http://docs.jj-vcs.dev/latest/glossary/>. Accessed August 2026.
- [61] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanping Li. 2018. Measuring program comprehension: A large-scale field study with professionals. *IEEE Transactions on Software Engineering* 44, 10 (2018), 951–976. doi:10.1109/TSE.2017.2734091
- [62] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [63] Annie T. T. Ying, Gail C. Murphy, Raymond Ng, and Mark C. Chu-Carroll. 2004. Predicting source code changes by mining change history. *IEEE Transactions on Software Engineering* 30, 9 (2004), 574–586. doi:10.1109/TSE.2004.52
- [64] YoungSeok Yoon and Brad A. Myers. 2014. A longitudinal study of programmers' backtracking. In *Proceedings of the 2014 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 101–108. doi:10.1109/VLHCC.2014.6883030
- [65] YoungSeok Yoon and Brad A. Myers. 2015. Supporting selective undo in a code editor. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*. 223–233. doi:10.1109/ICSE.2015.43
- [66] YoungSeok Yoon, Brad A. Myers, and Sebon Koo. 2013. Visualization of fine-grained code change history. In *Proceedings of the 2013 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 119–126. doi:10.1109/VLHCC.2013.6645254
- [67] Thomas Zimmermann, Peter Weißgerber, Stephan Diehl, and Andreas Zeller. 2004. Mining version histories to guide software changes. In *Proceedings of the 26th International Conference on Software Engineering (ICSE)*. 563–572. doi:10.1109/ICSE.2004.1317478